CHITKARA UNIVERSITY
HIMACHAL PRADESH

PROJECT REPORT

On

Multi-Tenant Coworking Management SaaS

BATCH-2024



Project Mentor

Mr. Baljeet Singh

Student Name & ID

1. Samyan Sharma (2411981646)
2. Arpit Khanal (2411981649)
3. Prakriti Shrestha (2411981652)
4. Nitesh Kumar Sah(2411981663)

Department of Computer Science and Engineering

Acknowledgement

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of our project, "**Multi-Tenant Coworking Management SaaS**"

First, we would like to thank our project mentor, **Mr. Baljeet Singh**, for his valuable guidance, continuous support, constructive suggestions, and encouragement throughout the project. His guidance helped us understand the technical requirements and improve the overall quality of our work.

We are also thankful to the **Department of Computer Science and Engineering, Chitkara University**, for providing us with the necessary resources, facilities, and learning environment required to successfully complete this project.

We would like to express our gratitude to our faculty members for their valuable feedback and technical guidance during the development and documentation of the project.

Finally, we would like to thank our team members for their cooperation, teamwork, and dedication. We also extend our appreciation to our family and friends for their constant encouragement and support throughout the project.

Team:

- Samyan Sharma (2411981646) .....
- Arpit Khanal (2411981649) .....
- Prakriti Shrestha (2411981652) .....
- Nitesh Kumar Sah (2411981663) .....

Abstract

Without any real-time visibility of availability or revenue, and with a process that grows cumbersome and inefficient with increased use, spreadsheets, chats and manual bank transfers are the typical way that independent coworking operators coordinate shared office space. In this project, we will share a multi-tenant coworking management platform that will replace these ad-hoc workflows and provide a web application as a backend that will feature three different roles: a cross-tenant Platform Administrator, a self-service Space Manager, and a booking Member.

The system is designed as a true multi-tenant application, that is, it has but one application instance and database, not one per each tenant. Tenant isolation is done at the application layer by re-verifying a JWT-based identifier on each request to return a 404, not a 403, when trying to access a resource on another tenant to prevent disclosure of the existence of the resource. This is achieved not by application-level checks (which are prone to concurrent requests) but rather by a PostgreSQL exclusion constraint which makes the fact that two bookings conflict structurally impossible at the database level. They are paid via Stripe Connect destination charges and are automatically split between 95/5, and payments are automatically retried and self-healed in case of payment reconciliation errors. Real-time floor-plan changes are sent through Socket.io, while a complete soft deletion workflow with cascading refunds is used to preserve the integrity of the historical data.

In addition to feature delivery, the project also identified and addressed six real security and reliability flaws before it was submitted, through an architecture review conducted through the project's self-discipline.

Table of Contents

1. Introduction 6
1.1 Objective 6
1.2 Technology Stack 7
2. Project Description 9
2.1 Problem statement: 9
2.2 Roles and permissions 9
2.3 Business Rules 10
2.4 Core Features 11
2.5 Scope Boundaries & Known Limitations 12
3. System Architecture & Design 13
3.1 High-Level System Architecture 13
3.2 API Request Lifecycle 15
3.3 Detailed System Design 16
3.4 Security Mechanics: 17
3.5. UML Diagrams 19
4. Database Design 20
4.1 Entity-Relationship Diagram (ERD) & Schema Overview 20
4.2 Relationships and Cardinality 21
5. Data Flow Diagrams 22
5.1 Level 0 Data Flow Diagram (Context Diagram) 22
5.2 Level 1 Data Flow Diagram (The Booking Journey) 23
6. Features Implemented 25
7. Screenshots 26
8. Challenges Faced 31
9. Future Enhancements 33
10. Conclusion 34
11. Appendix 35
References 36

Table of Tables

Table 1: Role definitions and key capabilities for the RBAC system. 9
Table 2: Detailed System Design..... 16
Table 3: Features implemented, mapped to their enforcing mechanism. 25
Table 4: Summary of API Endpoints, HTTP Methods, and Role-Based Access Control (RBAC) Permissions..... 35

Table of Figures

Figure 1: High-level layered architecture: client, API gateway, services, database. 13
Figure 2: Class diagram of the six core schema entities. 19
Figure 3: Entity-relationship diagram with cardinalities and cascade rules. 20
Figure 4: Level 0 Data Flow Diagram (Context Diagram). 22
Figure 5: Level 1 Data Flow Diagram 23
Figure 6: Login/Sign up Page Interface. 26
Figure 7: Member view: public directory for browsing and joining spaces. 26
Figure 8: Member view: live floor plan with real-time desk/room availability. 27
Figure 9: Member view: Stripe payment form for a booking held pending payment. 27
Figure 10: Member view: booking history with space, zone, and status. 28
Figure 11: Manager view: zone management, adding a desk with hourly/daily rates. 28
Figure 12: Manager view: Stripe Connect onboarding for payouts. 29
Figure 13: Manager view: revenue, bookings, and desk/room-split charts. 29
Figure 14: Admin view: cross-tenant growth, revenue, and top-spaces charts. 30
Figure 15: Admin view: drilling into a single tenant's zone. 30

1. Introduction

1.1 Objective

A backend-first web platform that lets an independent coworking operator manage an office space that is shared among a variety of tenants and users, with live availability, database-enforced booking integrity, and automated split payments. It replaces the spreadsheets, chat threads and manual bank transfers that are often used now by independent operators.

The platform is designed as an actual multi-tenant application. All spaces are considered separate tenants and common application instances and common databases are used by all tenants, not individual deployments. This is an important and intentional design decision, which was discussed at great length in Chapter 3, as it is much more cost-effective to operate and maintain, and much easier to design applications around, at the expense of having to carefully and wisely design isolation logic in the application layer, instead of relying on separation in the infrastructure layer.

There are three roles in the RBAC system:

- **Platform Administrator** - not scoped to any single space. Views platform-wide metrics across all tenants and can drill into any individual space's zones, members, and transaction history. Accounts are seeded by the development team; there is no self-service signup path for this role, since granting platform-wide visibility on demand would be a serious security risk.
- **Space Manager** - a self-service signup that creates exactly one Space in the same database transaction as the manager's own user account, so a manager account can never exist without an owning space, and vice versa. Manages zones, desks, and rooms within that space; sets hourly/daily booking price for each item in space; onboards to Stripe Connect to receive payouts; views the space's own analytics.
- **Member** - can join an existing Space by searching a public list of spaces currently active. Books desks and rooms and pays per booking with embedded Stripe Payment Element.

The purpose of the system built to demonstrate is to meet the following core engineering goals:

- Enforcing multi-tenant data isolation at the application layer with a re-validated tenant identifier (based on a JSON Web Token) on each request, not trusted at face value. [8][9]
- A database-based locking mechanism to prevent double booking. The system does not rely on application logic to perform checks for overlapping times before adding a new booking, since this logic can be easily fooled by concurrent requests; instead, it takes advantage of one of the specific PostgreSQL constraints that the database itself will enforce atomically at the time of the booking insertion.[3][4][13]
- Real-time updates of booking state, so that if one member books a desk, all other browsers that are currently viewing that particular floor plan will see the booking

change within one second or two seconds without having to refresh the browser page.[7]

- Automated marketplace payments that divide one single customer payment between the space company and the platform automatically, without the platform having to individually invoice each space company, with the help of Stripe Connect.[6]
- A defensive data lifecycle design that does not delete tenant data (space, booking and payment history) permanently from the database. Destructive actions are instead represented as soft deletion and cancellation, and there is an audit trail.

1.2 Technology Stack

The system is implemented as a TypeScript-first monorepo, where two applications, apps/backend and apps/frontend, have a shared repository for the benefit of both sets of developers to work off a shared source of truth (even if there's no formally generated API contract).

Frontend

- Next.js 16, using the App Router [11]
- React 19
- Tailwind CSS 4 for styling
- Zustand 5 for client-side state management, including persisting the authenticated user's JWT to the browser's local storage.[8][9]
- The socket.io-client library, for subscribing to real-time floor plan updates [7]
- The Stripe React SDK (@stripe/react-stripe-js), used to render the embedded Payment Element that collects card details in the browser without the application ever handling raw card numbers [6]

Backend

- NestJS 11, a modular, decorator-based Node.js framework that organizes code into controllers, services, and guards [1]
- Prisma ORM version 7, used with the @prisma/adapter-pg driver adapter, which allows Prisma to issue queries through the standard PostgreSQL driver rather than its own bundled query engine binary [2]
- PostgreSQL 16, run through Docker Compose during development, with the btree_gist extension enabled specifically to support the booking exclusion constraint described in Chapter 3 [3][4]
- JSON Web Tokens, issued using the jsonwebtoken library with an eight-hour expiry and no refresh token mechanism, combined with bcryptjs for one-way password hashing [8][9][10]

- Socket.io 4, configured so that each coworking space corresponds to exactly one Socket.io room, and a connecting client authenticates with the same JWT used for REST requests [7][8]
- The Stripe Node SDK, using Stripe Connect with the newer Accounts v2 API surface, Stripe's embedded Payment Element for card collection, and Stripe webhooks for asynchronous payment confirmation [6]

Testing

- Jest for the NestJS backend
- Vitest for the Next.js frontend

Tooling

- Docker Compose, used to run a local PostgreSQL instance during development
- git format-patch and git am as the team's internal convention for delivering and reviewing code changes, in preference to direct pushes

2. Project Description

2.1 Problem statement:

Most independent coworking spaces are operated with spreadsheet data and informal chat threads. This creates three recurring issues: desks and meeting rooms may end up being double-booked, there is no single source of truth on who is paying for a desk, and nobody (operator or prospective member) can see what is available at any given time. This platform solves all three problems by providing any operator a shared system in which members can view a real-time floor plan, book and pay online; operators will the get paid automatically via Stripe, and the platform operator will be able to view the utilization and revenue of every space on the platform.[6][7]

2.2 Roles and permissions

Table 1: Role definitions and key capabilities for the RBAC system.

Role	Scope	Key capabilities
Platform Administrator	Cross-tenant	Views platform-wide metrics and lists every space; can drill into any individual space to see its zones, members, and a paginated transaction history. Seeded account only; no self-service signup; cannot join real-time booking rooms.
Space Manager	Exactly one Space	Self-service signup creates the Space in the same transaction as the manager account; manages zones, desks, and rooms; sets a hourly/daily booking price; onboards to Stripe Connect; views the space's own analytics. May rename or soft-delete zones, desks, and rooms, and the deletion cascades to cancel and refund affected bookings.
Member	Joins one Space	Joins by browsing a public directory of active spaces. Books and pays for desks and rooms through an embedded card payment form. Cannot view analytics. Cannot cancel a booking independently; this is an explicit, deliberate scope decision rather than an oversight, discussed further in section 2.4.

2.3 Business Rules

The following are treated as product requirements rather than implementation details, meaning that any implementation approach is acceptable as long as the observed behavior satisfies the rule:

- A Space Manager can simultaneously hold a single LIVE Space only. Once closed, they can make only one new space. This is enforced atomically on the server, so that no two almost concurrent requests can cause a manager to hold two spaces simultaneously.
- No data is ever lost from the system databases. When deleting a Space, Zone, Desk, or Room, they will be identified as deleted instead of deleted from the system and all bookings and payment history will be maintained for reporting and audit.
- If the Zone is deleted, then the desks and rooms inside the zone are deleted as well - the soft delete. To delete a Space will propagate down to its Zones and beyond.
- All destructive actions have to be confirmed twice, with a "Delete" action before a specific "Are you sure?" prompt. A closed space also needs to be named exactly by the manager as an added protection from accidental irreversible action.
- If a resource is deleted while it has any upcoming bookings attached to it, it will cause a cancellation and refund process to run for those bookings – with an explicit warning to the manager displaying what will happen. When a book is deleted, no effect is felt on its previous bookings. Bookings which were not paid or were still being paid are cancelled without any refund, as no payment had been made for those bookings.
- If they do get refunds, they are full refunds. The customer gets 100% of the amount back, the 5% commission goes back to the platform, and the 95% withheld by the manager is returned. The original transaction's processing fee, which is charged by Stripe, cannot be recovered as Stripe cannot refund. If the refund fails (e.g., because of a transient error from Stripe's API), then the manager can try again, and the underlying deletion will not be rolled back until the refund is retried again.[6]
- Closed space members have their user account but are not able to access the member dashboard, instead they will see a dedicated "This space has been closed" screen. No way for a member to return to another space during the current series of the product.
- The revenue split for each booking is always 95 percent to the Space Manager and 5 percent goes to the platform, using Stripe Connect's destination charge mechanism, which is automatically applied for each booking, and is always in Stripe's test mode.
- The amount of double-booking is prevented not only by client-side validation that a determined user can break, but it is structurally impossible because of a constraint in the database that prevents it from happening.[3][4][13]
- A booking's price is the price as of the time the booking is booked. Any bookings created prior to the Space Manager changing the space's price are not affected.

- A booking in 'Waiting to pay' state will be sitting in its time slot for 15 minutes. If payment is not completed during that interval, then the slot is released back to availability, and the payment attempt in progress is cancelled.

2.4 Core Features

- A multi-tenant hierarchy of Space, Zone and Desk/Room, and tenant isolation enforced via JWT-derived identity and guards on the application-layer.[1][8][9]
- A space is self-signed up by a manager and a member is self-signup to a space with real JSON Web Token authentication and 3-roles RBAC.[1][8][9]
- A real-time live floor plan, built on Socket.io with a room for each space, updated as bookings are made in real time from all connected clients.[7]
- A booking flow using date and time pickers that visually block out booked slots, and a database level overlap constraint and 15-minute payment hold on newly created unpaid bookings.[3][4][13]
- Manager onboarding, automatic 95/5 destination charges, self-healing status reconciliation that corrects the stored status for a payment, which may have fallen out of sync with Stripe's own record at any point in the past, and an ability to retries a failed payment.[6]
- Space level analytics including revenue, active bookings, total bookings, utilization %, 30-day indicators and a zone breakdown of bookings.[6]
- Manager account settings for the space's booking cost and the status of Stripe's onboarding process.
- Soft delete and rename of zones, desks, and rooms, and the cascading cancel and refund behaviour outlined in Business Rule 5 (where a resource connected to the deleted room is also deleted from the room's floor plan), and the disappearance of the deleted resource from the connected room's floor plan for the connected client.
- A cancel and refund mechanism along with re-try support and refund status to the manager.
- Platform overview with active space counts, closed space counts, member counts across all tenants, payments by status, gross revenues, net revenue without refunds, net revenue, platform earnings and a trailing thirty-day net revenue. This last number isn't explicitly referred to as "Monthly Recurring Revenue" or "MRR" in the product, as there is no subscription billing for this system and to use those terms would be misleading about what you're measuring.
- Correct, real-time member, desks and room count displayed on each space's summary card.

- A public directory that allows a member to join a space without having to receive an invite code directly from the manager; both ways of joining space are used simultaneously.
- A Platform Administrator drill-down view that will enable the Platform Administrator to open the individual space and view: the list of its member(s) and the list of transactions, with a paginated history.

2.5 Scope Boundaries & Known Limitations

- Space closure and recreation - a Space Manager closing their current space and initializing exactly one replacement is the one piece of the originally scoped work carried forward into the next phase; it is detailed under Future Enhancements (Section 9) rather than repeated here.
- Out of scope for this version of the product, but would be required for a true public launch: Cancellations or refunds initiated by the member versus a resource deletion; subscriptions, memberships, or recurring revenue; more than one manager with a live space simultaneously; reopening a space after it has been closed; member reopen flow after space is closed; email or SMS notifications of any kind; Stripe's live payment mode, payment disputes, multiple currencies, or tax handling; partial refunds; pricing per desk vs. space; a manager approval step before the new member can join; mobile applications; single sign-on; or password reset or email verification flows.

3. System Architecture & Design

Multi-Tenant Coworking Management SaaS is a client-server application with three layers: Presentation Layer, Business Logic Layer and Database Layer. The architecture is designed for scalability, security, maintainability and high performance, and is able to accommodate multiple coworking organizations on a single platform. The frontend is built with React.js, the backend services with NestJS/Node.js, the database access with Prisma ORM, the database with PostgreSQL, and the real-time communication with Socket.io, as well as the secure payment processing with Stripe Connect.[1][2][6][7][11]

3.1 High-Level System Architecture

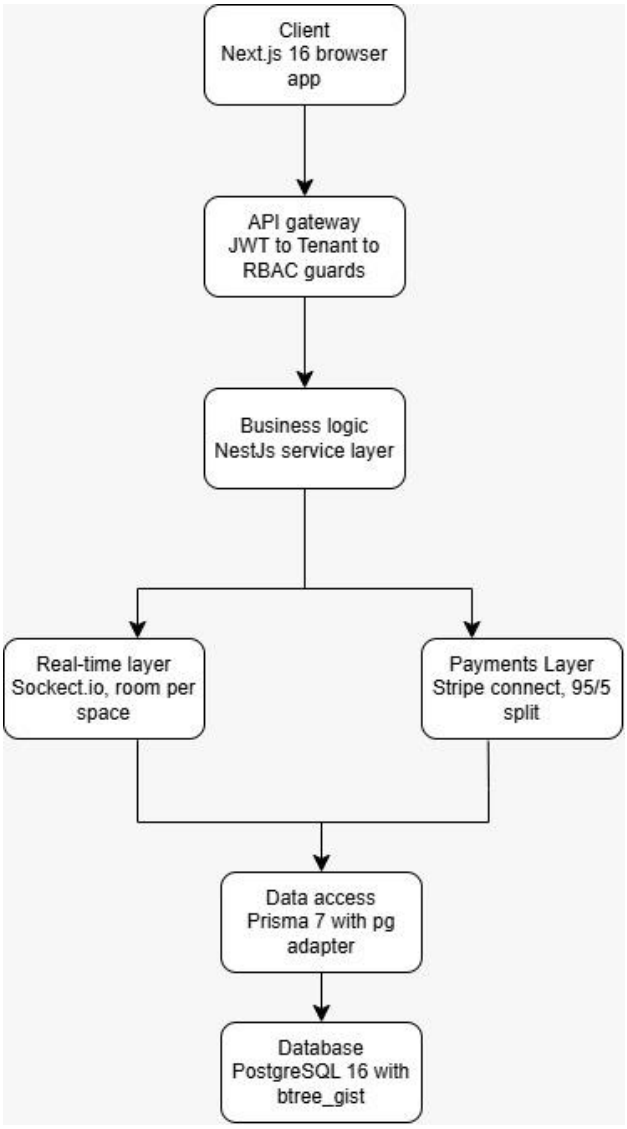


Figure 1: High-level layered architecture: client, API gateway, services, database.

The system doesn't need to use a microservices architecture: it's an appropriate and defensible choice for a single small team, a single deployable backend, and a workload that doesn't currently need different parts of functionality to scale independently. At this scale of a project, a microservice split would add distributed-systems complexity, including network calls

between services and eventual consistency, but without any added benefit, and would be a bad use of the short term of a single semester.

There are two properties of this architecture which merit more detailed explanation: they are the most technically interesting points which were taken in the system.

1. The Actual Location of Tenant Isolation

The database is a multi-tenancy model, which is usually referred to as **shared-schema, shared-database multi-tenancy**: there is only one PostgreSQL database, and each tenant's rows are in the same tables as the other tenants' rows but only differ by a spaceId foreign key column. This is a method that is not the same as two other popular methods, and it's important to understand why they were not selected:

- **Database-per-tenant rejection:** Provides the highest level of isolation guarantee, but is not scalable to hundreds or thousands of tenant database provisions, runs, and backups.
- **Schema-per-tenant rejection:** A middle ground that still involves per-tenant schema management overhead and isn't quite as powerful in terms of application logic as the shared-schema approach.
- **PostgreSQL Row-Level Security (RLS) rejection:** RLS would have prevented the database from returning any rows that didn't match a session policy. However, this was not adopted because it adds significant operational complexity that wasn't justified, given the team's limited experience with PostgreSQL policies and the critical project timeline.[5]

Application-Level Implementation & Security Rationale

Rather than relying on database-level features, isolation is implemented at the application level inside the NestJS application:

- **TenantGuard Execution:** Executed on every request, this guard extracts the spaceId from the claims in the caller's verified JWT token when accessing Space Manager and Member roles.[1][8][9]
- **Relationship Graph Re-checking:** This alone is not enough; each service method accessing tenant-scoped data separately re-checks ownership by traversing the actual relationship graph in the data store (e.g., checking that desk.zone.spaceId matches the caller's spaceId before returning details).
- **HTTP 404 vs. 403 Rationale:** The system intentionally returns an **HTTP 404 Not Found** response instead of **HTTP 403 Forbidden** on a cross-tenant access attempt. A 403 response leaks confirmation that a resource exists, revealing internal identifiers or what resources another tenant has. A 404 response provides no such confirmation, ensuring a member of one space cannot use the API to probe for resources in another space.

2. The Truth About Booking-Conflict Prevention

Booking concurrency is not handled via a client-side check or a simple "before insert row" layer check done by the NestJS service. Both of these strategies suffer from potential race conditions: if two requests to book the same desk for overlapping periods arrive within a sufficiently small window, the application might allow both to pass their checks before either completes its insert, causing a double booking.[12]

Instead, the real conflict check is handled directly at the database engine level:

- **Exclusion Constraint:** Built using an exclusion constraint on the Booking table using the EXCLUDE USING gist syntax alongside the btree_gist PostgreSQL extension, defined on a time range via startTime and endTime.[3][4][13]
- **Atomic Enforcement:** At the time of insertion, PostgreSQL refuses to commit a row if its time range overlaps with any other row in the same desk or room. This is enforced **atomically** as part of a transaction block, eliminating any possible race conditions.[4][13]
- **NestJS Service Role:** The only role of the NestJS service layer is to catch the returned PostgreSQL error and transform it into a clean **HTTP 409 Conflict** response for the client, preventing raw database errors from leaking.[1]

3.2 API Request Lifecycle

To make the architecture concrete, let us trace a single incoming request to book a new desk via a POST /bookings endpoint through the application pipeline:

1. **Authentication (JWT Verification):**
   - The member's JWT is passed via the Authorization: Bearer <token> header to the NestJS application.[8][9]
   - The default JwtAuthGuard runs as a global guard, verifying the token's signature and expiration period against the server's JWT secret.[1][8][9]
   - If verification fails, the request is denied immediately with a **401 Unauthorized** response, and no downstream handler code is executed.
2. **Tenant Isolation (TenantGuard):**
   - Next, the TenantGuard extracts the spaceId claim from the verified token and attaches it to the request object as req.spaceId.[1][8]
   - This provides a single, trusted source of truth for downstream code to identify the active tenant context.
3. **Role-Based Access Control (RbacGuard):**
   - The RbacGuard runs third, comparing the required role(s) of the route with the role claim embedded in the token.[1][8]
   - If the caller lacks the required permissions, the request is denied with a **403 Forbidden** response.
4. **Controller & Service Delegation:**

- Only after successfully passing all three guards does the request reach the BookingsController, which delegates execution to the BookingsService.[1]

5. **Database Transaction & Atomic Conflict Check:**
   - The service initiates a database transaction to attempt insertion of the new booking row.
   - As detailed in Section 3.1, the database-level exclusion constraint either permits the insert or atomically rejects it to prevent double booking.[3][4][13]
6. **Stripe Payment Intent Generation:**
   - If the insert succeeds, the service generates a Stripe PaymentIntent utilizing Stripe Connect's destination charge mechanism (incorporating a 5% platform application fee).[6]
   - It returns the new booking object along with a client secret required by the frontend to process payment.
7. **Secure Frontend Payment Execution:**
   - The frontend uses the client secret to embed Stripe's isolated Payment Element, capturing card information directly within Stripe's secure environment.[6]
   - The application backend never handles or stores raw card data.
8. **Webhook Confirmation & State Synchronization:**
   - Upon payment confirmation, Stripe asynchronously triggers a webhook call to the backend endpoint.[6]
   - The service updates the booking's paymentStatus field to reflect the completed transaction.
9. **Real-Time Floor Plan Broadcast:**
   - Finally, a booking state change event is broadcast to all clients currently viewing that room on the floor plan via WebSockets, ensuring synchronized UI updates across all active browsers without requiring manual page refreshes.[7]

3.3 Detailed System Design

Table 2: Detailed System Design

Module	Responsibility
Authentication	Dual-mode signup, where a manager's signup transactionally creates a Space and a Member's signup joins an existing one; password hashing with bcrypt; JWT issuance [8][10]
Space Management	Space creation, update, and soft deletion; slug generation with retry on collision; hourly/daily per-item pricing
Zone Management	Zones as the organizational unit within a Space, such as an open workspace area, private cabins, or a meeting room area

Resource Management	Desks and Rooms, each scoped to a Zone rather than directly to a Space, so that tenant scoping for a desk is always derived by following desk.zone.spaceId
Booking	Availability checking, a fifteen-minute payment hold, and database-enforced conflict prevention as described in section 3.1 [3][4][13]
Payments	Stripe Connect onboarding, 95/5 destination charges, webhook handling, and self-healing reconciliation of stale payment statuses [6]
Real-Time	Socket.io, with one room per Space and a JWT-authenticated connection handshake [7][8]
Lifecycle and Administration	Soft deletion of Space, Zone, Desk, and Room; the cancel-and-refund saga described in section 2.3; the Platform Administrator overview and per-space drill-down; the public space directory
Space Closure and Recreation	Allowing a Space Manager to close their space, see it removed from active use, and create exactly one replacement

A quick overview of the history of data lifecycle design and why it is important. Prior to this current design, deletion of a Space was a hard delete for both Space to Zone (and then Desk or Room) and Space to User. This would result in the deletion of a manager's account when deleting a space, and the deletion of every member's payment history since their payment history would only be recorded on Booking rows. This was determined to be a design error, and the patch that was applied, was a change to the relevant foreign keys on User.spaceId and Booking.userId, CASCADE deletion behavior, and RESTRICT – meaning that the database will no longer hard delete a Space if there are still users and bookings in it! The soft-delete-and-refund workflow is the one that's going to be supported going forward, and which is described above and confirmed.

3.4 Security Mechanics:

Several security-relevant mechanisms are worth documenting explicitly:

- **Password storage:** Never store passwords in plain text. Each password is salted and then one-way hashed during sign up, and the login just tries to compute the same hash of the password provided as it was during sign up and compares against the one-way hash, not the plain text password value.
- **Token-based authentication:** Stateless JSON Web Tokens are used for authentication instead of server-side sessions. This does not require any shared session store and makes horizontal scaling simpler (with the limitation that no refresh token mechanism or server-side token revocation list exists, meaning a token stays valid for its full 8-hour

duration even if the user changes their password). This is a documented and recognized limitation outlined in the Challenges chapter.

- **Payment data handling:** The application does not hold, pass, or store the raw card number of a member. It only ever includes a Stripe-generated token and client secret, running in a completely isolated Payment Element context controlled by Stripe.
- **Cross-tenant information leakage:** As mentioned in section 3.1, it is a deliberate design decision to return 404 instead of 403 on a cross-tenant access attempt, which acts as a security control to prevent an attacker from determining if resources exist for a different tenant.
- **Known gaps:** There are weaknesses that it is important to state with clarity rather than sweeping them under the carpet, as honesty about limitations demonstrates engineering maturity:
    - **Self-joining spaces:** A member can self-join any live space from the public directory with no manager approval step in between. This is a deliberate product-scope decision rather than an oversight.
    - **JWT storage:** JWTs are stored in the browser's localStorage (see Chapter 1's technology stack), which is readable by any script running on the page, meaning a successful XSS attack would expose a user's token. This is a known trade-off of the stateless-JWT approach chosen for simplicity; an httpOnly cookie would close this vector but was not implemented.

3.5. UML Diagrams

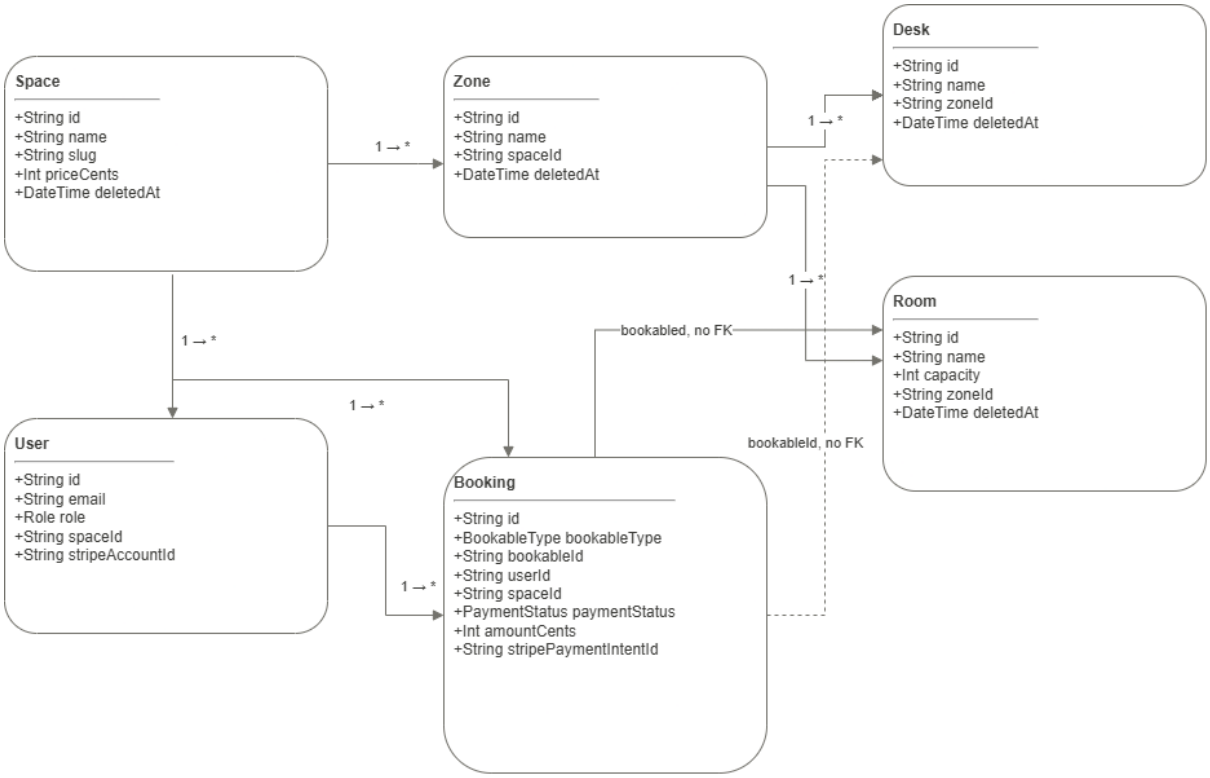


Figure 2: Class diagram of the six core schema entities.

4. Database Design

4.1 Entity-Relationship Diagram (ERD) & Schema Overview

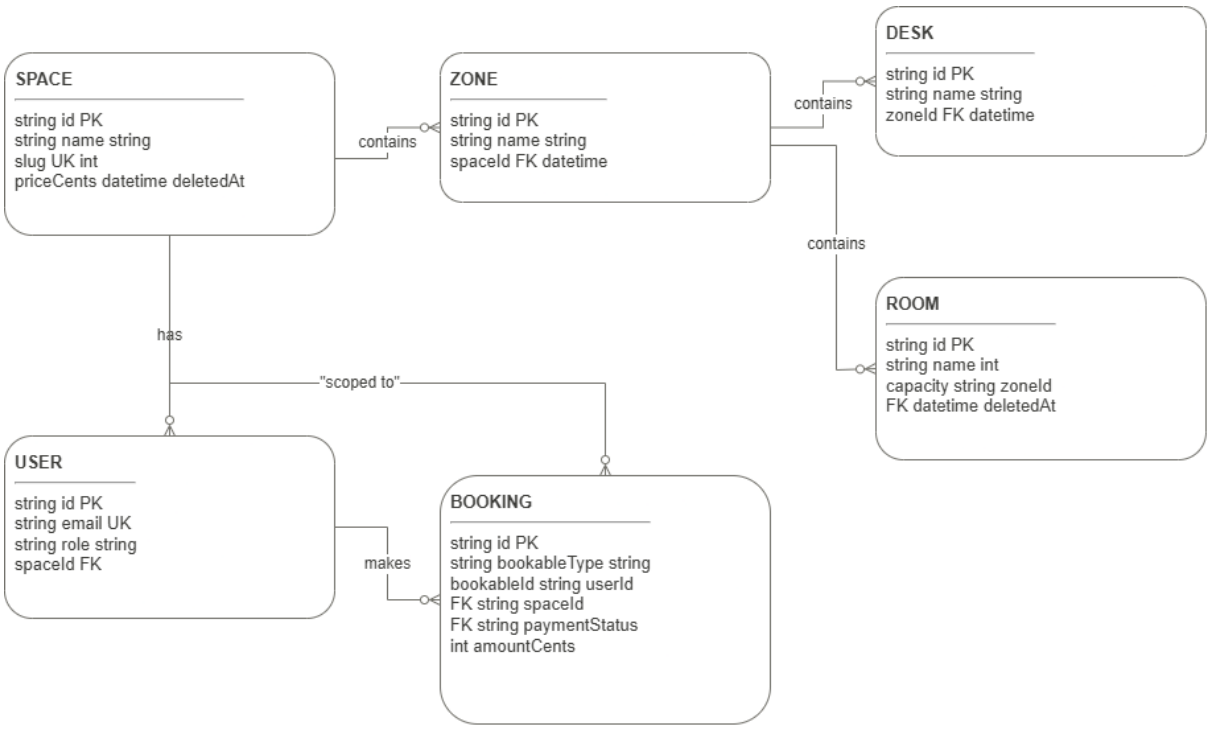


Figure 3: Entity-relationship diagram with cardinalities and cascade rules.

The database schema consists of **six core entities**. It is important to explicitly note architectural decisions regarding what *is not* in the schema: there is no separate Platform table and no standalone Payment table.

- **Platform Administrator:** The platform admin role is not an independent tenant entity; rather, it is handled via an enumeration value on the User entity.
- **Payments:** Payment details are not stored in a separate entity; instead, financial tracking attributes exist as direct fields on the Booking entity.

The schema entities comprise the following attributes:

- **Space:** Contains id, name, slug (unique), priceCents, and deletedAt.
- **Zone:** Consists of id, name, spaceId, and deletedAt.
- **Desk:** Consists of id, name, zoneId, and deletedAt.
- **Room:** Consists of id, name, capacity, zoneId, and deletedAt.
- **User:** Includes id, email (unique), password, name, role, spaceId, stripeAccountId, and stripeOnboardingComplete.
- **Booking:** Contains time and financial metadata, including holdExpiresAt, platformFeeCents, paidAt, cancelledAt, cancellationReason, stripeRefundId, and refundedAmountCents (along with foreign keys and structural identifiers).

4.2 Relationships and Cardinality

The relationships between entities govern data integrity, cascading rules, and architectural trade-offs:

- **Space to Zone (1-to-Many):** Managed with Zone.spaceId as a foreign key, featuring cascading deletion to automatically remove child zones when a parent space is deleted.
- **Zone to Desk & Room (1-to-Many):** Managed via Desk.zoneId and Room.zoneId foreign keys respectively.
- **Space to User (1-to-Many):** Utilizes User.spaceId as a foreign key. This relationship was intentionally implemented with **restrictive deletion**—the database prevents a space from being hard-deleted if any users are still attached to it, aligning with the safety measures outlined in Section 3.3.
- **User to Booking (1-to-Many):** Links users to their respective bookings via userId, behaving consistently regarding deletion constraints.
- **Space to Booking (1-to-Many):** Utilizes Booking.spaceId as a direct foreign key. This was added expressly as a snapshot reference of a space; for example, a space's revenue and booking history can always be calculated directly from this column without having to traverse through current (or potentially modified/deleted) spaces, desks, or rooms.

Polymorphic Resource Reference & Design Trade-off

- **bookableId and bookableType:** A booking contains a bookableId that can reference either a Desk.id or a Room.id, determined by the value of the bookableType field.
- **Design Trade-off:** There is intentionally no database-level foreign key constraint enforcing this polymorphic relationship. Creating two separate booking tables was evaluated as overly complex to build and reason about compared to a single unified Booking table. The notable trade-off is that the database engine cannot natively verify whether a given bookableId references a valid row, shifting this validation to the application layer.

Concurrency and Uniqueness Constraints

- **Uniqueness:** There is a strict unique constraint on the combination of bookableType, bookableId, and startTime.
- **Overlap Prevention:** As detailed in Chapter 3, a separate PostgreSQL exclusion constraint (EXCLUDE USING gist) ensures that no two bookings of the same resource can ever have overlapping time periods.[3][4][13]

5. Data Flow Diagrams

5.1 Level 0 Data Flow Diagram (Context Diagram)

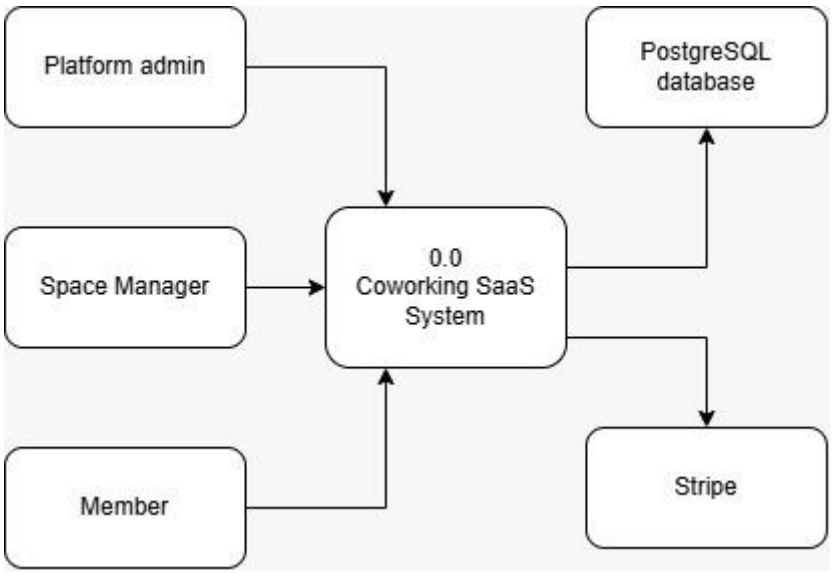


Figure 4: Level 0 Data Flow Diagram (Context Diagram).

The Level 0 Data Flow Diagram (DFD) models the system as a single, centralized process representing the multi-tenant coworking management application, interacting directly with three human actors and two external systems:

- Human Actors:
    - Platform Administrator: Sends administrative and oversight queries, receiving system-wide operational metrics and tenant overviews.
    - Space Manager: Sends space configuration, resource management, and billing setup commands, receiving onboarding statuses and analytics.
    - Member: Sends registration, directory browsing, space-joining requests, and desk/room booking queries, receiving real-time floor plan updates and checkout details.
- External Systems:
    - PostgreSQL Database Store: Exchanges read/write queries for persistent storage of tenants, spaces, zones, desks, rooms, users, and booking records.
    - Stripe Payment Gateway: Exchanges data for executing destination charges, managing marketplace splits between operators and the platform, and processing refunds and webhooks.[6]

5.2 Level 1 Data Flow Diagram (The Booking Journey)

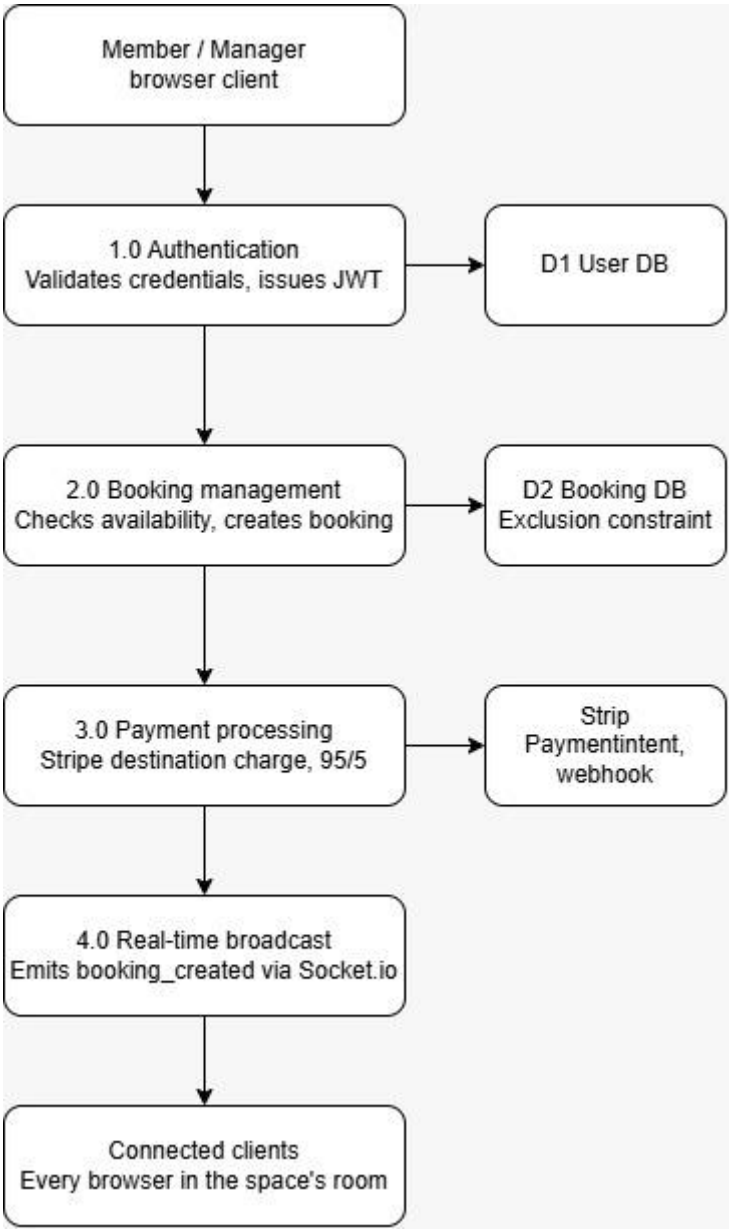


Figure 5: Level 1 Data Flow Diagram

The single process from the Level 0 context diagram is decomposed in the Level 1 DFD to zoom into the core booking journey. This granularity provides the most value walkthrough, breaking the workflow down into four primary processes:

1. Authentication Process:
    - Function: Verifies member or manager credentials against the User data store.
    - Data Flow: Reads user records, verifies password hashes, and issues a signed JSON Web Token (JWT) containing role and spaceId claims back to the actor.[8][9][10]

2. Booking Management Process:
    - Function: Handles resource selection and temporal conflict resolution.
    - Data Flow: Interacts with the Booking and Resource data stores, leveraging the PostgreSQL exclusion constraint to atomically verify availability and insert new booking records without race conditions.[3][4][13]
3. Payment Processing Process:
    - Function: Manages financial transactions and webhook state synchronization.
    - Data Flow: Interfaces with the external Stripe API to initiate destination charges (applying the 95/5 marketplace revenue split) and listens for asynchronous webhooks to update booking payment statuses.[6]
4. Real-Time Broadcast Process:
    - Function: Synchronizes UI states across active client sessions.
    - Data Flow: Transmits booking state change events via Socket.io to all web browsers actively viewing that specific floor plan room.[7]

Scope Note: Ancillary processes such as general Space Management, Zone Management, and basic Resource CRUD are intentionally excluded from this specific diagram to prevent visual clutter and maintain clarity during academic evaluation and viva presentations.

6. Features Implemented

Table 3: Features implemented, mapped to their enforcing mechanism.

Feature	Description
Multi-tenant architecture	Tenant isolation enforced at the application layer through a JWT-derived space identifier, independently re-verified through the relation chain on every request, with a 404 response rather than a 403 on cross-tenant access. [1][8][9]
Role-based access control	Three roles, each gated at the level of individual routes through a dedicated guard.[1]
Public space directory	A member browse a directory of active spaces.
Real-time floor plan	Booking availability updates instantly across all connected clients using Socket.io.[7]
Database-enforced conflict prevention	A PostgreSQL exclusion constraint is the authoritative mechanism preventing double-booking, independent of any application-level check.[3][4][13]
Booking holds	A booking awaiting payment reserves its slot for fifteen minutes before automatic release
Stripe Connect payments	Destination charges with an automatic 95/5 split between the space manager and the platform.[6]
Payment retry and reconciliation	A failed or stalled payment can be retried, and stale payment statuses are self-corrected against Stripe's own record.[6]
Space analytics	Revenue, booking counts, utilization rate, and thirty-day trend indicators
Booking and payment history	Members and managers can view past and upcoming bookings together with payment details
Soft deletion and renaming	Zones, desks, and rooms may be renamed or soft-deleted, with two-step confirmation and a cascading cancel-and-refund process for affected bookings
Platform Administrator overview and drill-down	Cross-tenant metrics, and the ability to open any individual space to see its zones, members, and transaction history
Responsive interface	Built with Next.js and Tailwind CSS for consistent behavior across desktop, tablet, and mobile screen sizes.[11]

7. Screenshots

This chapter presents the user interface of the running application, captured from the main branch.

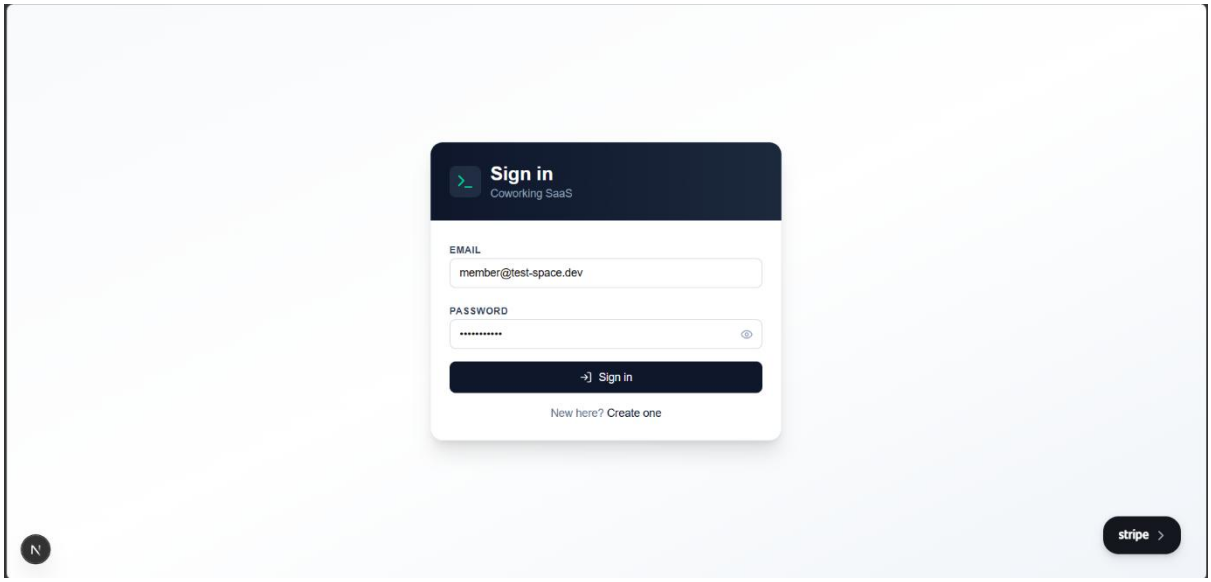


Figure 6: Login/Sign up Page Interface.

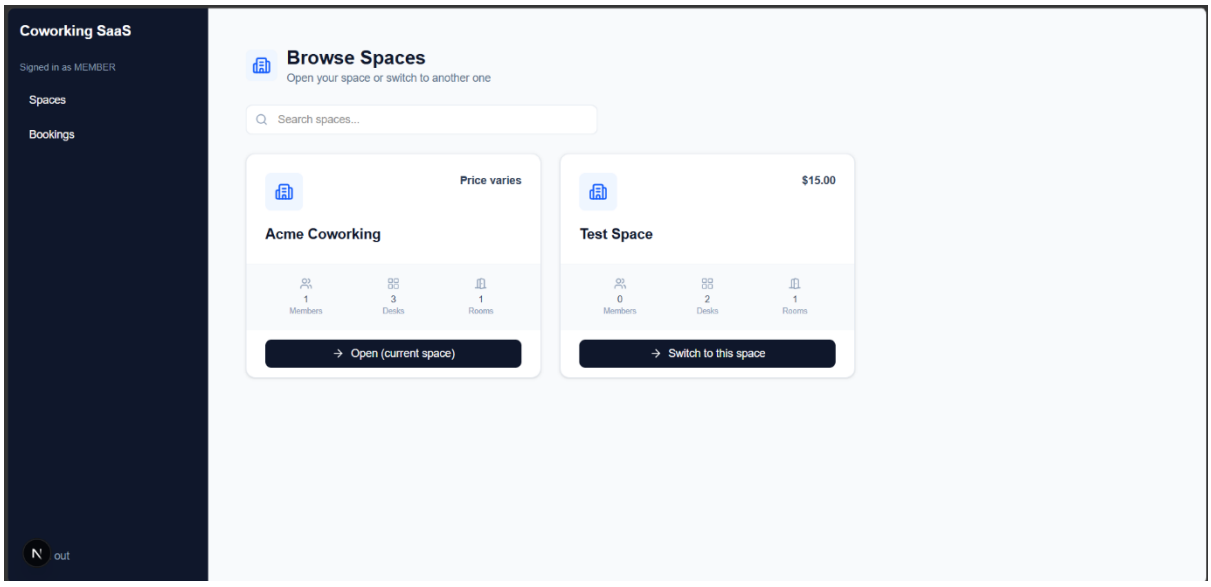


Figure 7: Member view: public directory for browsing and joining spaces.

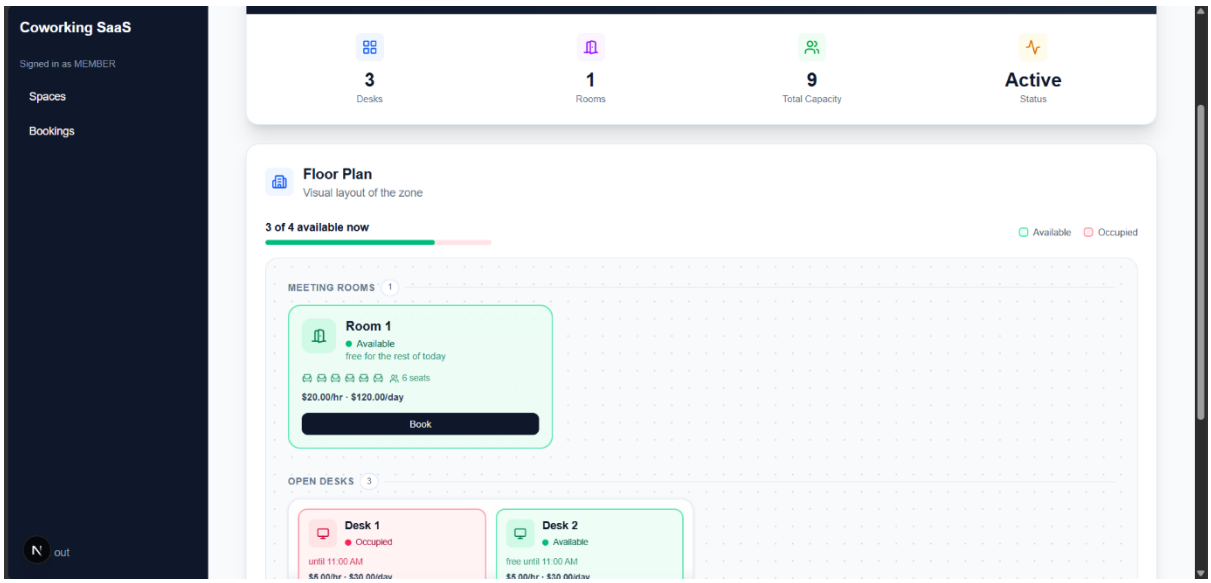


Figure 8: Member view: live floor plan with real-time desk/room availability.

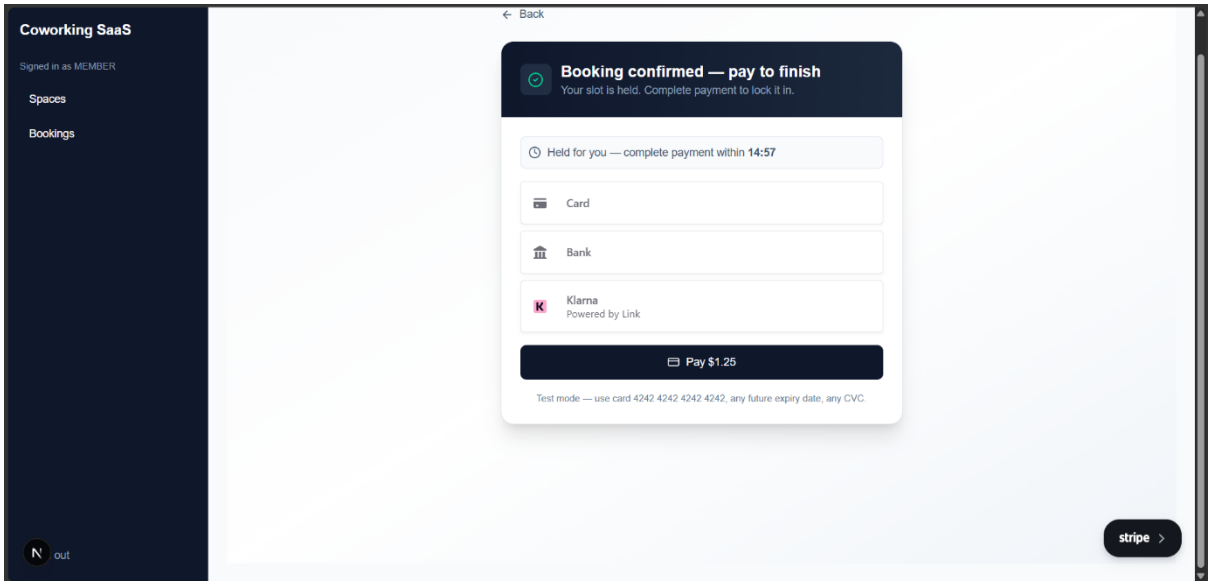


Figure 9: Member view: Stripe payment form for a booking held pending payment.

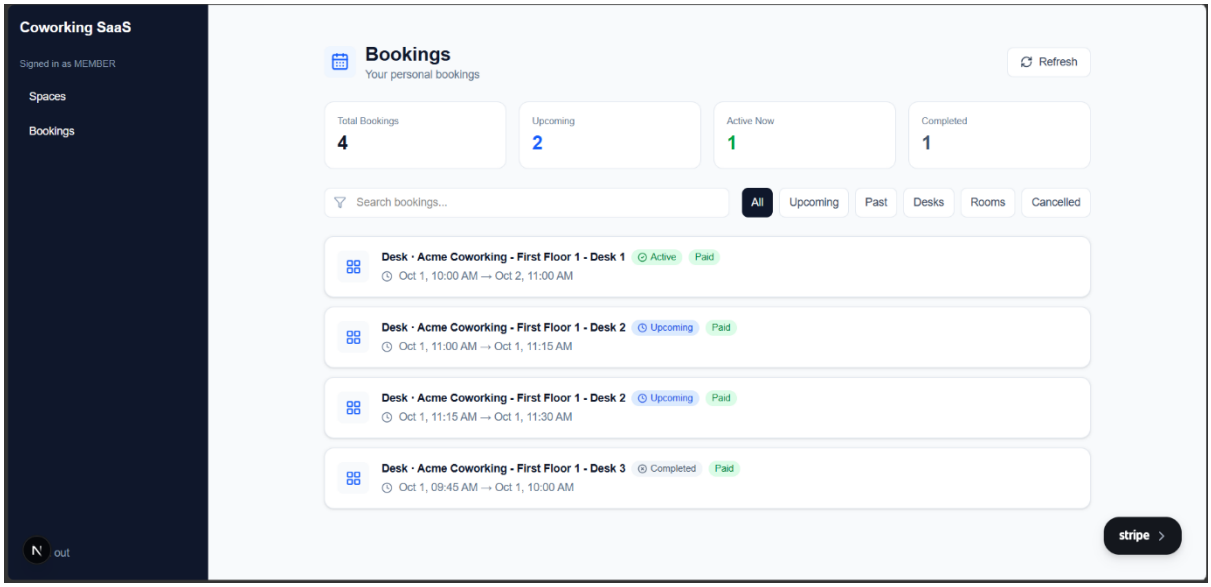


Figure 10: Member view: booking history with space, zone, and status.

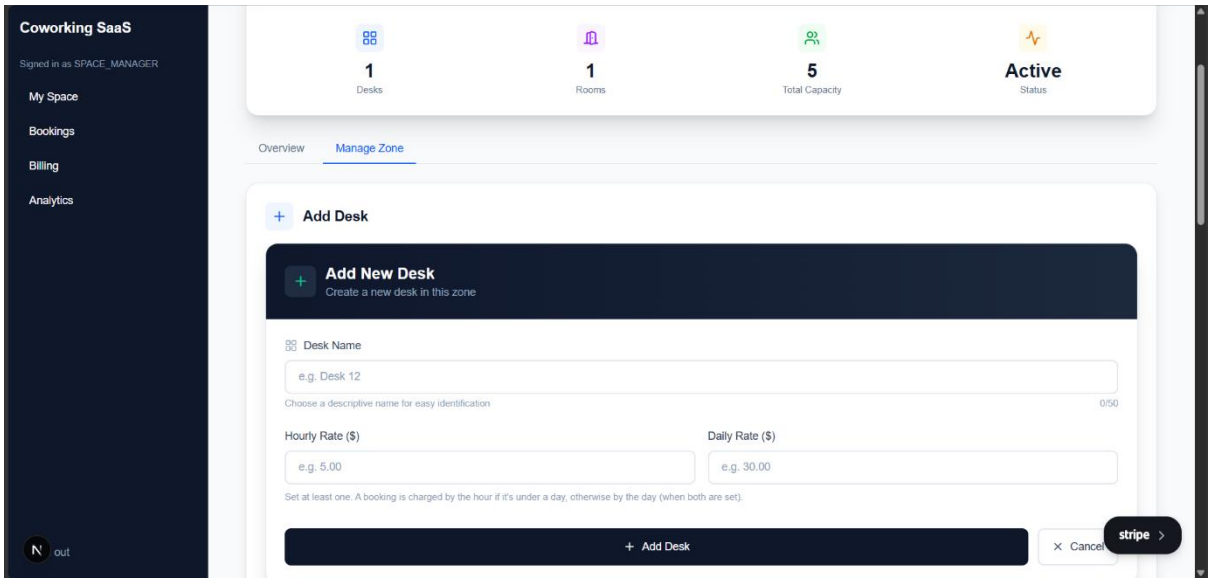


Figure 11: Manager view: zone management, adding a desk with hourly/daily rates.

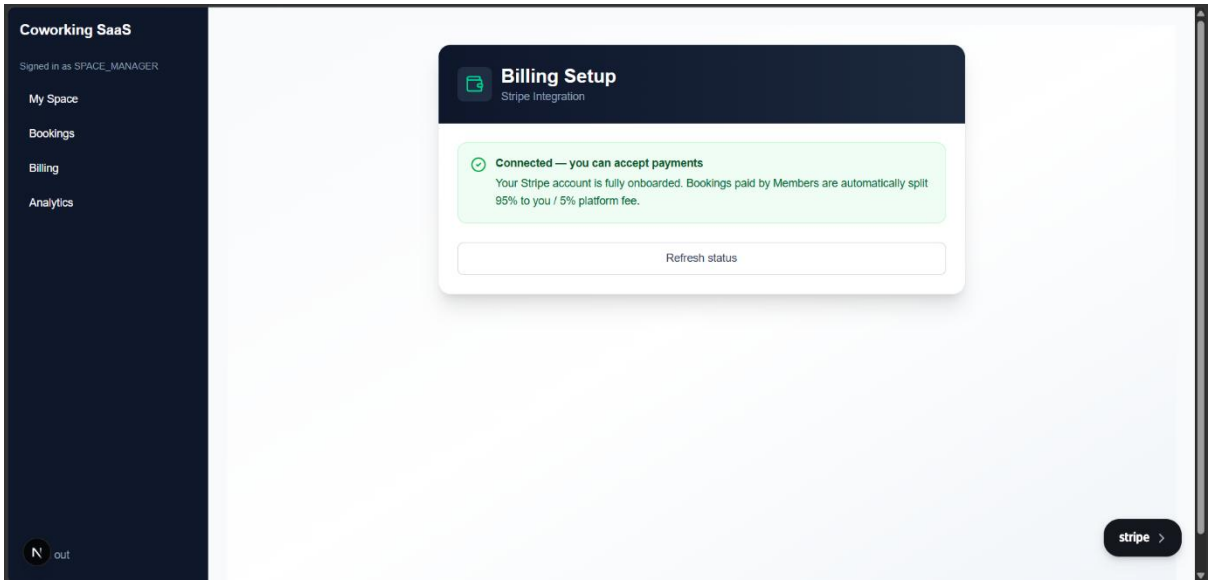


Figure 12: Manager view: Stripe Connect onboarding for payouts.

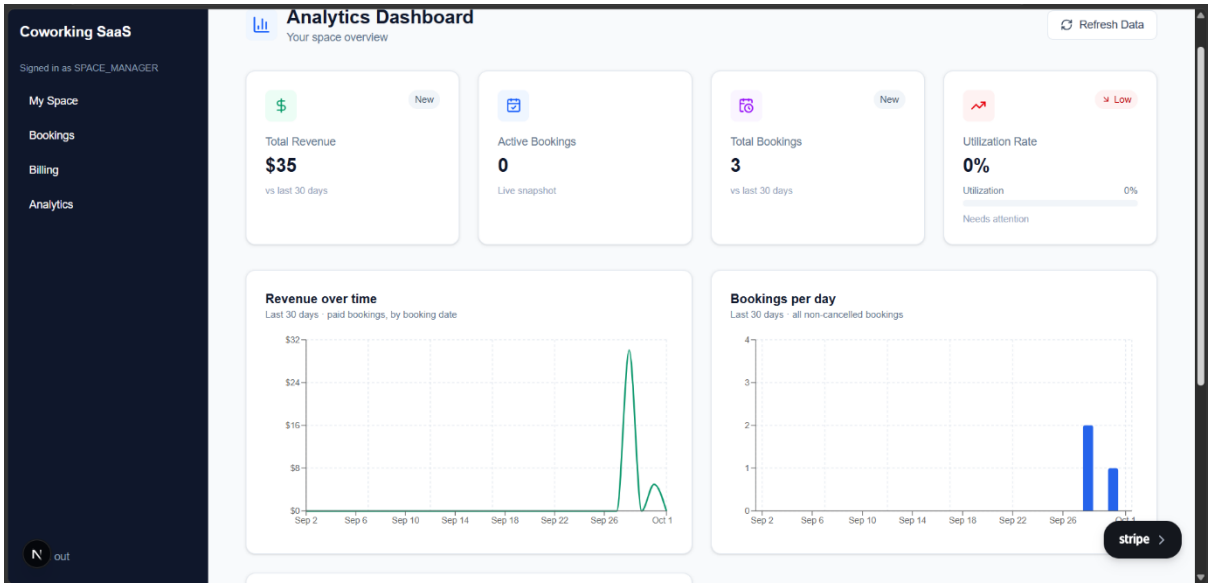


Figure 13: Manager view: revenue, bookings, and desk/room-split charts.

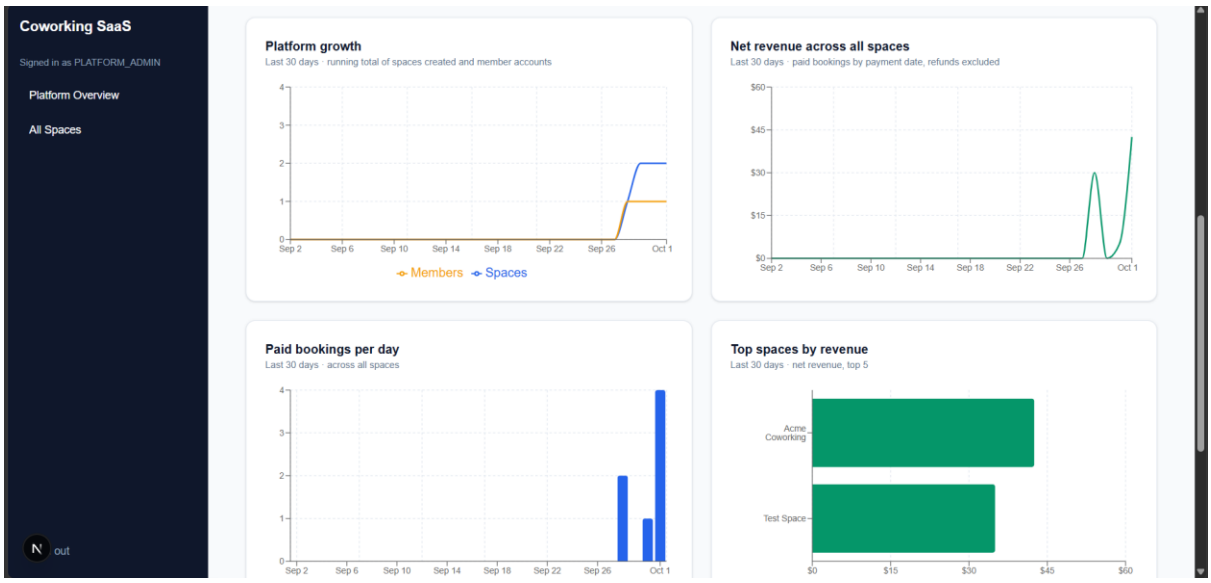


Figure 14: Admin view: cross-tenant growth, revenue, and top-spaces charts.

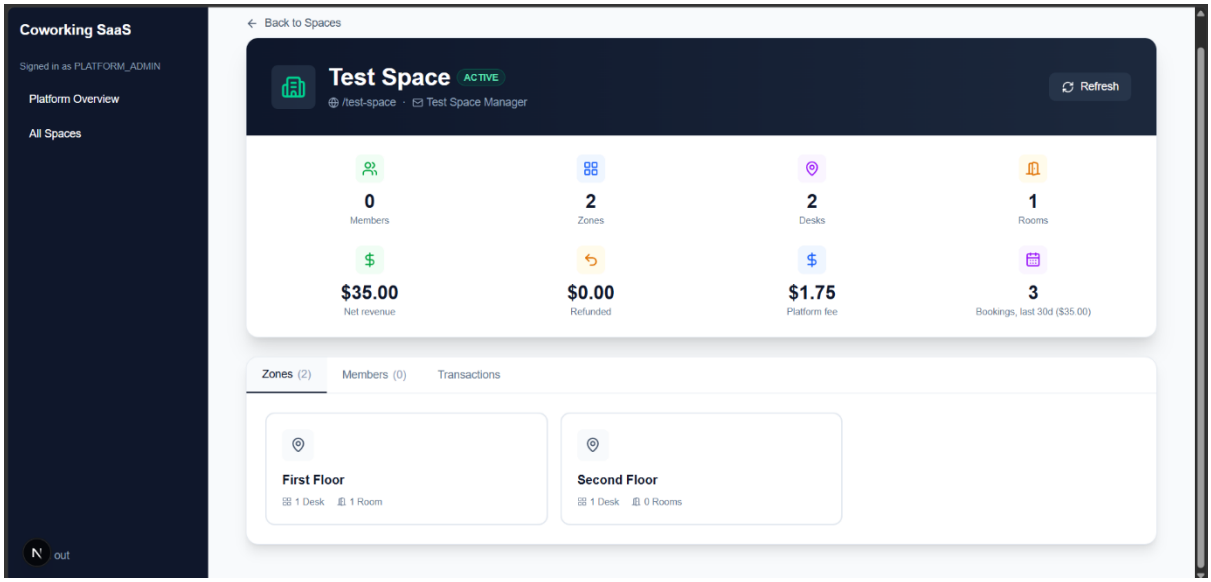


Figure 15: Admin view: drilling into a single tenant's zone.

8. Challenges Faced

During the design, development, and integration of the Multi-Tenant Coworking Management SaaS project, the team encountered several significant engineering hurdles. Documenting these challenges and their resolutions demonstrates the technical maturity and iterative problem-solving required to build a production-grade system.

1. Enforcing Tenant Isolation Without Database-Level Row Security

- **The Challenge:** The team chose not to use PostgreSQL's built-in Row-Level Security (RLS) due to operational complexity and timeline constraints. Instead, tenant isolation relied entirely on application-level defenses.
- **The Resolution:** The team established a rigorous two-layer application guard architecture: a TenantGuard to extract tenant identity from the verified JWT token, combined with mandatory re-verification inside every service method by walking the relationship chain of the requested resource. This required strict coding discipline across all services to prevent any isolation gaps from reopening.

2. Maintaining Single Booking Integrity Under Concurrent Requests

- **The Challenge:** Initial implementations utilized an unsafe "check-then-insert" availability pattern. Under realistic race conditions, two concurrent requests could simultaneously pass the validation check and successfully insert overlapping bookings, causing double bookings.
- **The Resolution:** The logic was shifted out of the application code entirely and delegated to the database engine. The team implemented a PostgreSQL exclusion constraint using the btree_gist extension, leveraging the database's native transactional guarantees to ensure correctness atomically.

3. Overcoming Stripe Connect Onboarding Restrictions

- **The Challenge:** Mid-development restrictions on the platform's Stripe dashboard temporarily blocked the full onboarding and payment execution flow.
- **The Resolution:** Rather than halting development on core booking features, the team decoupled the workflow. Bookings could be created in an unpaid state and completed later via a separate payment endpoint once the underlying Stripe account configuration issues were resolved.

4. Preserving Financial Reporting During Soft Deletions

- **The Challenge:** Early soft-delete implementations allowed a desk or room to be marked deleted while removing it from current records. This orphaned historical bookings, causing them to drop out of revenue analytics because queries relied on a live join to the current desk or room record.

- **The Resolution:** This was resolved by introducing a direct, stable Booking.spaceId foreign key. All historical and analytical queries were refactored to read directly from this snapshot column instead of performing live joins through potentially deleted resources.

5. Restricting Real-Time Payload Scope for Privacy

- **The Challenge:** Early versions of real-time booking events and list endpoints broadcasted a booking's full database row to all space members including sensitive data such as the paying member's user identifier and raw Stripe payment intent identifiers.
- **The Resolution:** Discovered during internal code reviews, this was addressed by sanitizing the broadcast payload shape to expose only non-sensitive details visible to non-owning members.

6. Managing Parallel Development Sessions and Git Hygiene

- **The Challenge:** Uncoordinated parallel development sessions resulted in a defective commit being merged directly into the main branch.
- **The Resolution:** The team updated their workflow conventions, mandating a strict protocol of running git fetch and inspecting recent commit logs before beginning any new coding session to prevent working on stale or conflicting local states.

9. Future Enhancements

While the Multi-Tenant Coworking Management SaaS project has successfully shipped a robust set of features to the main branch, several technical enhancements, configuration adjustments, and remaining scope components are identified for future deployment and product maturity:

- **Production Deployment Strategy:** A final hosting environment has not yet been selected, leaving the application running exclusively in a local development environment.
- **Schema-Level Tenant-Isolation Backstop:** Desk and Room currently have no spaceId column of their own—a caller's tenant is verified by walking up through Zone.spaceId in each service method by hand. Adding a spaceId column to both tables, checked through one shared helper, would make a forgotten check in a future endpoint fail safely instead of silently leaking across tenants.
- **GET /bookings Read Optimization:** GET /bookings currently does write work on every call it expires stale holds and reconciles every PENDING booking's status against Stripe as part of serving a read request. This keeps the list self-healing but makes reads slower, non-idempotent, and a source of avoidable Stripe API calls; moving both into a scheduled background job (keeping the webhook as the primary update path) would make listing a pure read.
- **Polymorphic Foreign-Key Constraint Trade-off:** The polymorphic Booking.bookableId has no foreign-key constraint by necessity it points at either a Desk or a Room row depending on bookableType, which Prisma/Postgres cannot express as a single foreign key. Referential integrity here is enforced entirely in application code rather than the schema, representing a deliberate and documented architectural trade-off.
- **Booking-Overlap Exclusion Constraint vs. Cancelled Bookings:** The booking-overlap exclusion constraint does not currently exclude cancelled bookings from the overlap check. This is harmless today because a booking is only ever cancelled as a side effect of its desk or room being soft-deleted, but it would need to be revisited if member-initiated cancellation is introduced later.

10. Conclusion

The Multi-Tenant Coworking Management SaaS showcases a fully functional prototype of a production-oriented multi-tenant backend system. Running on the project's main branch, the system successfully integrates real JWT-based authentication, application-enforced tenant isolation with an information-safe error response pattern, database-authoritative guarantees against double-booking, live state synchronisation over Socket.io, and automated Stripe Connect marketplace payments all of which are fully verifiable.

A significant secondary feature set has also been completed and merged, including safe and auditable soft deletions, comprehensive cancel-and-refund workflows with retry support, platform-wide administrative views, per-space analytics drill-downs, and a public space directory. The only remaining item under the original approved scope is the implementation of space closure and recreation.

Key Verified Achievements

- **Application-Layer Multi-Tenant Isolation:** Informed and intentional design decisions, such as returning secure 404 error codes instead of 403 to prevent data leakage regarding other tenants' resources.
- **Race-Condition-Free Booking Concurrency:** Database-authoritative conflict prevention leveraging PostgreSQL exclusion constraints rather than vulnerable application-level logic.
- **Real-Time State Synchronization:** Sub-second UI updates propagated across concurrently connected clients using Socket.io rooms.
- **Automated Marketplace Payments:** Streamlined revenue splitting (95/5) via Stripe Connect, reinforced with automated self-healing reconciliation and retry mechanisms.
- **Defensive Data Lifecycles:** Full soft-delete and refund pipelines paired with a platform-wide administrative overview.

Engineering Value & Reflection

The true engineering value of this project lies in making specific, defensible implementation decisions grounded in proof rather than assumption. By addressing architectural challenges at the database layer (such as concurrency and structural constraints) rather than relying solely on application-level checks, and by treating soft deletion as a first-class schema requirement from inception rather than an afterthought, the system avoids common pitfalls of rapid software development. Maintaining an honest and continuously updated internal record of what is proven to work—while acknowledging known limitations—demonstrates the core maturity required for scalable, enterprise-grade software engineering.

11. Appendix

GitHub Repository: https://github.com/SamyanSharma/Multi-Tenant-Coworking-Management-SaaS

Selected API surface, all routes requiring a Bearer JWT except where marked public:

Table 4: Summary of API Endpoints, HTTP Methods, and Role-Based Access Control (RBAC) Permissions

Method	Path	Roles
POST	/auth/login, /auth/signup	Public
GET	/spaces/public	Public
GET	/spaces, /spaces/me, /spaces/:id	Administrator or any authenticated role
POST	/zones, /desks, /rooms	Manager
GET	/bookings	Manager, Member
POST	/bookings, /bookings/:id/pay	Member
POST	/payments/onboard; GET /payments/status	Manager
POST	/payments/webhook	Public, verified by Stripe's request signature
GET	/analytics/summary, /analytics/bookings-per-zone	Manager, Administrator
GET	/admin/spaces/:id, /admin/spaces/:id/bookings, /admin/spaces/:id/members	Administrator

References

[1] NestJS, "Guards," NestJS Documentation. [Online]. Available: https://docs.nestjs.com/guards. [Accessed: Sep. 30, 2026].
[2] Prisma, "Database drivers and driver adapters," Prisma Documentation. [Online]. Available: https://docs.prisma.io/docs/orm/v7/core-concepts/supported-databases/database-drivers. [Accessed: Sep. 30, 2026].
[3] PostgreSQL Global Development Group, "F.9. btree_gist — GiST operator classes with B-tree behavior," PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/btree-gist.html. [Accessed: Sep. 30, 2026].
[4] PostgreSQL Global Development Group, "5.4. Constraints," PostgreSQL 16 Documentation, including Section 5.4.6, "Exclusion Constraints." [Online]. Available: https://www.postgresql.org/docs/16/ddl-constraints.html. [Accessed: Sep. 30, 2026].
[5] PostgreSQL Global Development Group, "5.9. Row Security Policies," PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/ddl-rowsecurity.html. [Accessed: Sep. 30, 2026].
[6] Stripe, "Connect: Destination Charges," Stripe Documentation. [Online]. Available: https://docs.stripe.com/connect/destination-charges. [Accessed: Sep. 30, 2026].
[7] Socket.IO, "How it works," Socket.IO Documentation, Version 4. [Online]. Available: https://socket.io/docs/v4/how-it-works. [Accessed: Sep. 30, 2026].
[8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519. [Accessed: Sep. 30, 2026].
[9] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token Best Current Practices," RFC 8725, Internet Engineering Task Force, Feb. 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8725. [Accessed: Sep. 30, 2026].
[10] OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: Sep. 30, 2026].
[11] Next.js, "App Router," Next.js Documentation. [Online]. Available: https://nextjs.org/docs/app. [Accessed: Sep. 30, 2026].
[12] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.
[13] PostgreSQL Global Development Group, "CREATE TABLE," PostgreSQL 16 Documentation, including exclusion constraint definitions. [Online]. Available: https://www.postgresql.org/docs/16/sql-createtable.html. [Accessed: Sep. 30, 2026].